

Reproduction of Paper: SARS-CoV-2 restructures host chromatin architecture

Author 1, Author 2, and Author 3

2026-07-06

Table of contents

1	Introduction	1
2	Analyses	2
2.1	Part 1: AB Compartment Analyses	2
2.2	Part 2: TAD Analyses	11
2.3	Part 3: Chromatin Loop Analyses	25

1 Introduction

In this project, we aim to re-analyze and reproduce parts of the paper: “SARS-CoV-2 restructures host chromatin architecture” by Wang, R., Lee, JH., Kim, J. et al., published in Nature Microbiology in 2023. This paper aims to characterize the 3D genome and epigenome of human cells after SARS-CoV-2 infection, where they find widespread host chromatin restructuring and show that SARS-CoV-2 affects host chromatin. Before this study, it was relatively unknown whether SARS-CoV-2 restructures host chromatin. This study uses a range of bioinformatic approaches and methods to investigate that, some of which are brought up in class. As COVID-19 was a recent large-scale pandemic, we wanted to apply the tools we have learned to study the implications of SARS-CoV-2. In our analyses, we focus on altered TAD contacts, chromatin compartmentalization, and loop extrusion, supporting the authors’ conclusions that SARS-CoV-2 restructures host chromatin.

2 Analyses

A/B Compartment Analyses

1. Load Hi-C Files

Similar to the TAD analyses, we used the cooler tool (version 0.10.4) to load the Hi-C files to prepare them for downstream analyses. The `cooler merge` commands merge two 80 kb resolution Hi-C contact matrices for the infected (`merged_cov_HiC.mcool`) and mock-treated (`merged_mock_HiC.mcool`) conditions into separate merged `.mcool` files. The `cooler balance` commands then perform matrix balancing on each merged dataset. We restricted our analysis to chromosome 9 at 80kb resolution to reproduce Fig 1c in particular, and it also helps in reducing computational complexity.

```
cooler merge merged_cov_HiC.mcool \
  GSM5411175_NS74-NS67-HiC7.mcool::/resolutions/80000 \
  GSM5411176_NS74-NS67-HiC8.mcool::/resolutions/80000

cooler merge merged_mock_HiC.mcool \
  GSM5411173_NS74-NS67-HiC1.mcool::/resolutions/80000 \
  GSM5411174_NS74-NS67-HiC2.mcool::/resolutions/80000

cooler balance merged_mock_HiC.mcool
cooler balance merged_cov_HiC.mcool
```

2. GC track for eigenvector phasing

As described in the paper, A/B nuclear compartments were identified on the basis of decomposed eigenvectors (E1) from Hi-C contact matrices using cooltools. A/B compartmental scores (E1) were corrected by GC densities in each bin.

To do so, we used cooltools (version 0.7.1, the paper used version 0.4.1 but it is unsupported on our system, and the results are reproduced) and downloaded the hg38 reference. The paper used the hg19 reference, but we managed to reproduce the result with a different reference.

We generated the genomic annotation files at 80 kb resolution on chromosome 9. We calculated chromosome sizes for the hg38 reference genome and cleaned the file to retain only standard chromosomes with numeric lengths. The genome is then partitioned into 80 kb bins using `cooltools genome binnify`, and filtered for bins corresponding to chromosome 9. Because different tools use different chromosome naming conventions, we temporarily renamed `chr9` to the NCBI accession (CM000671.2) so that GC content can be calculated from the corresponding NCBI reference genome FASTA. After GC content is computed for each genomic bin, we converted the chromosome name back to the standard `chr9` format. We created a chromosome view file defining the genomic coordinates of chromosome 9 to provide a reference region for subsequent analyses (compartment calling or visualization).

```
cooltools genome fetch-chromsizes hg38 > hg38.chrom.sizes

# cleaning
awk 'BEGIN{OFS="\t"} $2 ~ /^[0-9]+$/ {print $1, $2}' \
  hg38.chrom.sizes > hg38.clean.chrom.sizes
cooltools genome binnify hg38.clean.chrom.sizes 80000 > hg38.80000.bins.bed

# filter for chromosome 9
awk '$1=="chr9"' hg38.80000.bins.bed > chr9.80000.bins.bed

# standardizing naming conventions
awk 'BEGIN{OFS="\t"} NR==1{$1="chrom"; print; next} \
  $1=="chr9"{$1="CM000671.2"; print}' \
  chr9.80000.bins.bed > chr9.80000.bins.ncbi.bed

cooltools genome gc chr9.80000.bins.ncbi.bed \
  ncbi_dataset/ncbi_dataset/data/GCA_000001405.15/GCA_000001405.15_GRCh38_genomic.fna \
  > chr9.80000.gc.tsv

# standardizing naming conventions
awk 'BEGIN{OFS="\t"} NR==1{print; next} {$1="chr9"; print}' \
  chr9.80000.gc.tsv > chr9.80000.gc.chrnames.tsv

echo -e "chr9\t0\t138394717\tchr9" > chr9.view.tsv
```

3. Compartment analysis

Saddle plot analyses were performed to measure the compartmentalization strength on a genome-wide scale using `cooltools`. We first used `cooltools eigs-cis` to perform eigenvalue decomposition to calculate compartment signal by finding the eigenvector that correlates best with the phasing track, using GC content to orient the compartments and producing

compartment tracks such as E1. Next, we used `cooltools expected-cis` to calculate the expected Hi-C signal for cis regions of chromosomal interaction maps, and then `cooltools saddle` using the mock E1 compartment ordering to generate saddle plots for both mock and CoV samples. This allows for comparison of A/B compartment strength and organization under the same reference ordering. Then, we used `cooler dump`, `cooler load`, and `cooler balance` to extract only chr9 bins and contacts from the mock dataset, rebuild them as a standalone `.cool` file, and normalize it.

```
for SAMPLE in mock cov; do
  cooltools eigs-cis \
    merged_${SAMPLE}_HiC.mcool \
    --view chr9.view.tsv \
    --phasing-track chr9.80000.gc.chrnames.tsv::GC \
    --n-eigs 3 \
    -o ${SAMPLE}_80000
done

for SAMPLE in mock cov; do
  cooltools expected-cis \
    --view chr9.view.tsv \
    merged_${SAMPLE}_HiC.mcool \
    -p 8 \
    -o ${SAMPLE}_chr9_80000.expected.tsv

  cooltools saddle \
    --view chr9.view.tsv \
    merged_${SAMPLE}_HiC.mcool \
    mock_80000.cis.vecs.tsv::E1 \
    ${SAMPLE}_chr9_80000.expected.tsv::balanced.avg \
    -n 50 \
    --qrange 0.02 0.98 \
    --fig png \
    --strength \
    -o ${SAMPLE}_chr9_80000.mockOrdered.saddle
done

cooler dump -t bins merged_mock_HiC.mcool | awk '$1=="chr9"' > chr9.bins.tsv
cooler dump -t pixels merged_mock_HiC.mcool --join \
  | awk '$1=="chr9" && $4=="chr9"{print $2"\t"$5"\t"$7}' > chr9.pixels.tsv

cooler load -f coo chr9.bins.tsv chr9.pixels.tsv merged_mock_chr9.cool
```

```
cooler balance merged_mock_chr9.cool
```

4. Correlation maps from observed/expected Hi-C contact matrices

We generated Pearson correlation maps from observed/expected Hi-C contact matrices for chromosome 9 in the mock and CoV conditions. We computed the expected cis contact frequencies for chr9, extracted the chr9 region from 100–130 Mb (for Fig 1c), and converted the matrix into an observed/expected matrix by normalizing each diagonal by its expected value. We then plotted the resulting correlation matrix.

```
# pearson_compartments.py

import cooler
import cooltools
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def corr_pairwise_nan(oe):
    n = oe.shape[0]
    corr = np.full((n, n), np.nan)

    for i in range(n):
        xi = oe[:, i]
        for j in range(i, n):
            xj = oe[:, j]
            valid = np.isfinite(xi) & np.isfinite(xj)

            if valid.sum() >= 3:
                corr_ij = np.corrcoef(xi[valid], xj[valid])[0, 1]
                corr[i, j] = corr_ij
                corr[j, i] = corr_ij

    return corr

def pearson_oe(clr_path, chrom, out_png):
    clr = cooler.Cooler(clr_path)

    view_df = pd.DataFrame({
        "chrom": [chrom],
        "start": [0],
```

```

        "end": [clr.chromsizes[chrom]],
        "name": [chrom],
    })

exp = cooltools.expected_cis(
    clr,
    view_df=view_df,
    nproc=4,
)

res = clr.binsize

start_bp = 100_000_000
end_bp = 130_000_000

mat = clr.matrix(balance=True).fetch(
    (chrom, start_bp, end_bp)
).astype(float)

chrom_exp = exp[exp["region1"] == chrom].set_index("dist")["balanced.avg"]

n = mat.shape[0]

oe = np.full(mat.shape, np.nan)

for d in range(n):
    val = chrom_exp.get(d, np.nan)
    if np.isfinite(val) and val > 0:
        i = np.arange(n - d)
        oe[i, i + d] = mat[i, i + d] / val
        oe[i + d, i] = mat[i + d, i] / val

corr = corr_pairwise_nan(oe)

plt.figure(figsize=(7, 6))
plt.imshow(
    corr,
    cmap="RdBu_r",
    origin="upper",
    extent=(100, 130, 130, 100),
)
plt.xlabel("Position (Mb)")

```

```

plt.ylabel("Position (Mb)")
plt.colorbar(label="Pearson r")
plt.title(f"{chrom} Pearson O/E")
plt.tight_layout()
plt.savefig(out_png, dpi=300)
plt.close()

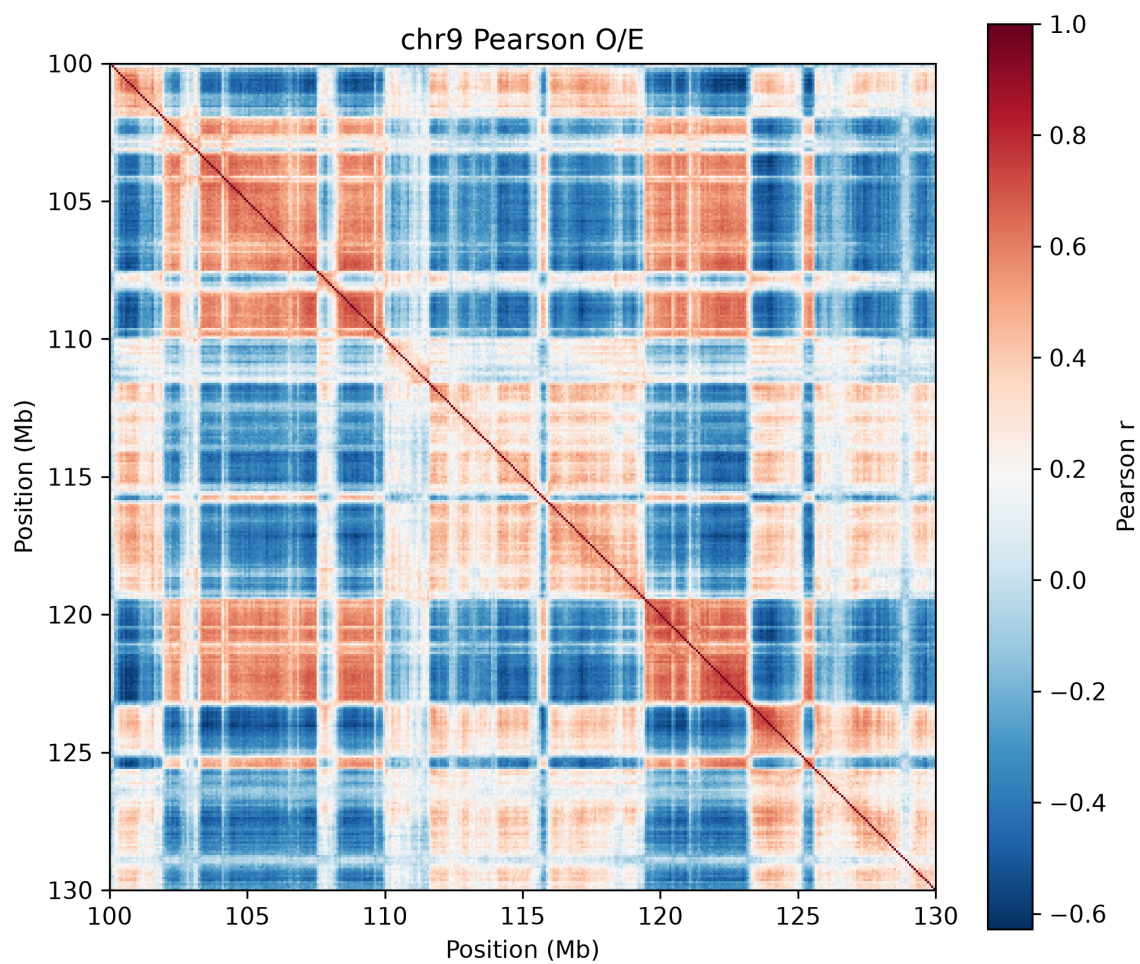
pearson_oe("merged_mock_HiC.mcool", "chr9", "mock_chr9_pearson.png")
pearson_oe("merged_cov_HiC.mcool", "chr9", "cov_chr9_pearson.png")

```

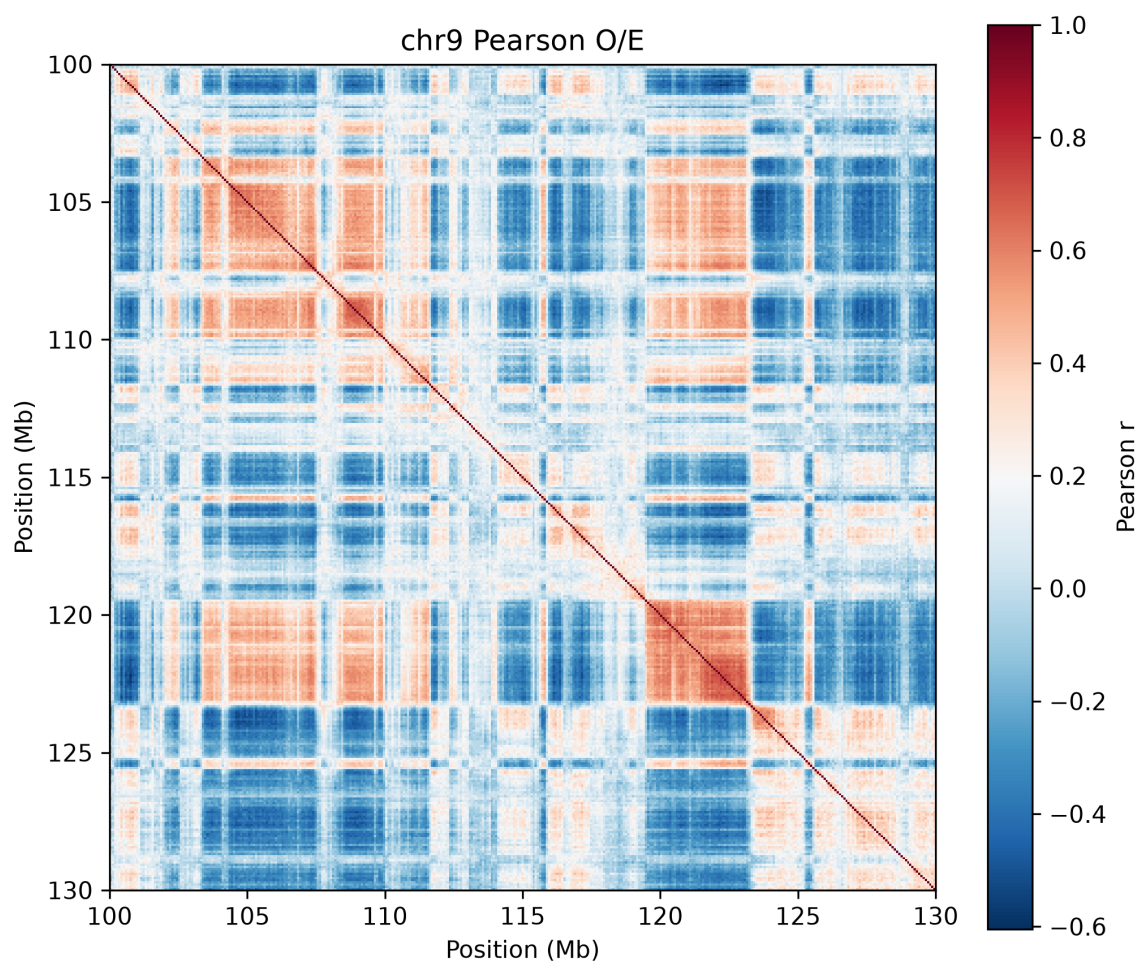
With reference to the figures below, we reproduced Fig 1c. Shown below are the Pearson correlation matrices of Hi-C of a region (chr9: 100–130 Mb, hg19), where they show alteration of the 3D genome and chromatin compartmentalization by SARS-COV-2. These matrices are used to visualize chromatin compartmentalization (A and B compartments) by measuring how similarly different genomic regions interact with the rest of the chromosome. From the matrices and the right panel of Fig 1c, we see that there is increased A-B mixing and reduced A-A interactions, indicating that interactions between active (A) and inactive (B) chromatin compartments become more similar after infection and there is a weakening of interactions among transcriptionally active chromatin regions. These observations demonstrate that SARS-CoV-2 disrupts large-scale chromatin organization by weakening compartment structure. Instead of maintaining distinct active and inactive chromatin domains, infected cells exhibit increased mixing between compartments and reduced cohesion within active chromatin

Our results differ from the paper's in one area: we obtained negative correlations in regions where the paper obtained zero correlations. We propose that this reflects differences in processing that the paper did not mention in detail. In our code, Pearson correlations are computed directly from the observed/expected matrix, so bins with opposite interaction patterns can produce negative values. The result from the paper may mask negative correlations or used a different normalization. We could also have used different balancing weights, expected-cis calculation, or differed in whether low-coverage bins were filtered. It would help if the authors could describe their method in greater detail to explain this discrepancy.

Pearson correlation matrix of Hi-C: Mock



Pearson correlation matrix for Hi-C: SARS-COV-2



Part2: TAD-Insulation and Intra-TAD Interactions

Xiaohan Xie

0. Overview

This section investigates how SARS-CoV-2 infection perturbs chromatin architecture at the level of topologically associating domains (TADs). Specifically, we reproduce three analyses from the original study:

Specifically, we aim to reproduce the following analyses in the original paper: 1. Boundary integrity: whether TAD boundary strength changes upon infection; 2. Intra-TAD interactions: whether contact frequency within TADs is altered; 3. Direct comparison of Hi-C contact matrices at a representative TAD domain.

```
library(HiCExperiment)
```

Consider using the ``HiContacts`` package to perform advanced genomic operations on ``HiCExperiment`` objects.

Read "Orchestrating Hi-C analysis with Bioconductor" online book to learn more:
<https://js2264.github.io/OHCA/>

```
library(HiContacts)
```

Registered S3 methods overwritten by 'readr':

method	from
as.data.frame.spec_tbl_df	vroom
as_tibble.spec_tbl_df	vroom
format.col_spec	vroom
print.col_spec	vroom
print.collector	vroom
print.date_names	vroom

```
print.locale          vroom
str.col_spec          vroom
```

```
library(GenomicRanges)
```

Loading required package: stats4

Loading required package: BiocGenerics

Loading required package: generics

Attaching package: 'generics'

The following objects are masked from 'package:base':

```
as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
setequal, union
```

Attaching package: 'BiocGenerics'

The following object is masked from 'package:HiCExperiment':

```
as.data.frame
```

The following objects are masked from 'package:stats':

```
IQR, mad, sd, var, xtabs
```

The following objects are masked from 'package:base':

```
anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
unsplit, which.max, which.min
```

Loading required package: S4Vectors

Attaching package: 'S4Vectors'

The following object is masked from 'package:HiCExperiment':

```
metadata<-
```

The following object is masked from 'package:utils':

```
findMatches
```

The following objects are masked from 'package:base':

```
expand.grid, I, unname
```

Loading required package: IRanges

Loading required package: Seqinfo

```
library(InteractionSet)
```

Loading required package: SummarizedExperiment

Loading required package: MatrixGenerics

Loading required package: matrixStats

Attaching package: 'MatrixGenerics'

The following objects are masked from 'package:matrixStats':

```
colAlls, colAnyNAs, colAnys, colAvgsPerRowSet, colCollapse,  
colCounts, colCummaxs, colCummins, colCumprods, colCumsums,  
colDiffs, colIQRDiffs, colIQRs, colLogSumExps, colMadDiffs,  
colMads, colMaxs, colMeans2, colMedians, colMins, colOrderStats,  
colProds, colQuantiles, colRanges, colRanks, colSdDiffs, colSds,
```

```
colSums2, colTabulates, colVarDiffs, colVars, colWeightedMads,
colWeightedMeans, colWeightedMedians, colWeightedSds,
colWeightedVars, rowAlls, rowAnyNAs, rowAnys, rowAvgsPerColSet,
rowCollapse, rowCounts, rowCummaxs, rowCummins, rowCumprods,
rowCumsums, rowDiffs, rowIQRDiffs, rowIQRs, rowLogSumExps,
rowMadDiffs, rowMads, rowMaxs, rowMeans2, rowMedians, rowMins,
rowOrderStats, rowProds, rowQuantiles, rowRanges, rowRanks,
rowSdDiffs, rowSds, rowSums2, rowTabulates, rowVarDiffs, rowVars,
rowWeightedMads, rowWeightedMeans, rowWeightedMedians,
rowWeightedSds, rowWeightedVars
```

Loading required package: Biobase

Welcome to Bioconductor

```
Vignettes contain introductory material; view with
'browseVignettes()'. To cite Bioconductor, see
'citation("Biobase")', and for packages 'citation("pkgname")'.
```

Attaching package: 'Biobase'

The following object is masked from 'package:MatrixGenerics':

```
rowMedians
```

The following objects are masked from 'package:matrixStats':

```
anyMissing, rowMedians
```

```
library(rtracklayer)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
x lubridate::%within%() masks IRanges::%within%()
x dplyr::collapse()      masks IRanges::collapse()
x dplyr::combine()       masks Biobase::combine(), BiocGenerics::combine()
x dplyr::count()         masks matrixStats::count()
x dplyr::desc()          masks IRanges::desc()
x tidyr::expand()        masks S4Vectors::expand()
x dplyr::filter()        masks stats::filter()
x dplyr::first()         masks S4Vectors::first()
x dplyr::lag()           masks stats::lag()
x ggplot2::Position()    masks BiocGenerics::Position(), base::Position()
x purrr::reduce()        masks GenomicRanges::reduce(), IRanges::reduce()
x dplyr::rename()        masks S4Vectors::rename()
x ggplot2::resolution() masks HiCExperiment::resolution()
x lubridate::second()     masks S4Vectors::second()
x lubridate::second<-()   masks S4Vectors::second<-()
x dplyr::slice()         masks IRanges::slice()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(epiwraps)
```

Loading required package: EnrichedHeatmap

Loading required package: grid

Loading required package: ComplexHeatmap

=====

ComplexHeatmap version 2.28.0

Bioconductor page: <http://bioconductor.org/packages/ComplexHeatmap/>

Github page: <https://github.com/jokergoo/ComplexHeatmap>

Documentation: <http://jokergoo.github.io/ComplexHeatmap-reference>

If you use it in published research, please cite either one:

- Gu, Z. Complex Heatmap Visualization. iMeta 2022.
- Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional genomic data. Bioinformatics 2016.

The new InteractiveComplexHeatmap package can directly export static complex heatmaps into an interactive Shiny app with zero effort. Have a try!

This message can be suppressed by:

```
suppressPackageStartupMessages(library(ComplexHeatmap))
=====
```

```
=====
EnrichedHeatmap version 1.42.0
Bioconductor page: http://bioconductor.org/packages/EnrichedHeatmap/
Github page: https://github.com/jokergoo/EnrichedHeatmap
Documentation: http://bioconductor.org/packages/EnrichedHeatmap/
```

If you use it in published research, please cite:
Gu, Z. EnrichedHeatmap: an R/Bioconductor package for comprehensive
visualization of genomic signal associations. BMC Genomics 2018.

This message can be suppressed by:
`suppressPackageStartupMessages(library(EnrichedHeatmap))`

```
=====
Warning: replacing previous import 'IRanges::median' by 'stats::median' when
loading 'epiwraps'
```

```
library(patchwork)
library(GenomicFeatures)
```

Loading required package: AnnotationDbi

Attaching package: 'AnnotationDbi'

The following object is masked from 'package:dplyr':

```
select
```

1. Data Preparation

Using the command line tool of cooler, we merged the replicates for Mock and SARS-CoV-2-infected conditions. In accordance with the original study's methods, we processed the data at 10kb resolution. Because the paper does not specify a normalisation procedure in detail, we applied cooler's built-in matrix-balancing algorithm to correct for coverage biases and generated balanced contact scores, which were used throughout the downstream analysis. Lastly, we performed matrix coarsening to generate multi-resolution data, as this was needed for the calculation of insulation scores.

We limited our analysis to chromosome 14 to reduce the computational complexity.


```
cooler merge merged_mock_HiC.mcool mock_HiC_rep1.mcool::/resolutions/10000 mock_HiC_rep2.mcool
cooler merge merged_cov_HiC.mcool cov_HiC_rep1.mcool::/resolutions/10000 cov_HiC_rep2.mcool:
cooler balance merged_mock_HiC.mcool
cooler balance merged_cov_HiC.mcool
cooler zoomify -r 10000,200000 merged_mock_HiC.mcool -o merged_mock_HiC_multi.mcool
cooler zoomify -r 10000,200000 merged_cov_HiC.mcool -o merged_cov_HiC_multi.mcool
```

```
hic_mock_merged <- HiCExperiment("./GSE179184_RAW/merged_mock_HiC_multi.mcool", focus = "chr14")
hic_cov_merged <- HiCExperiment("./GSE179184_RAW/merged_cov_HiC_multi.mcool", focus = "chr14")
length(hic_mock_merged)
```

```
[1] 3829954
```

```
length(hic_cov_merged)
```

```
[1] 3728822
```

```
summary(interactions(hic_mock_merged)$balanced)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
5.846e-05	2.257e-04	3.452e-04	1.193e-03	6.174e-04	2.909e-01	43969

```
summary(interactions(hic_cov_merged)$balanced)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
5.072e-05	2.495e-04	3.922e-04	9.968e-04	6.773e-04	1.201e-01	57439

Mock sample has slightly more interaction counts than the CoV-infected group. Both samples show highly skewed distribution, with the majority of interactions having very low balanced scores, which is expected for Hi-C data.

2. Insulation Scores and Boundary strengths

According to methods described in the original paper, we calculated the insulation scores using `getDiamondInsulation`, setting a 200kb sliding window. TAD boundaries were then called by finding the local minima in the insulation profile with a threshold of 0.5.

```
hic_mock <- getDiamondInsulation(hic_mock_merged, window_size = 200000)
```

Going through preflight checklist...

Scan each window and compute diamond insulation score...

Annotating diamond score prominence for each window...

```
hic_cov <- getDiamondInsulation(hic_cov_merged, window_size = 200000)
```

Going through preflight checklist...

Scan each window and compute diamond insulation score...

Annotating diamond score prominence for each window...

```
hic_mock <- getBorders(hic_mock, strong_threshold = 0.5)
hic_cov <- getBorders(hic_cov, strong_threshold = 0.5)
bounds_mock <- topologicalFeatures(hic_mock, 'borders')
bounds_cov <- topologicalFeatures(hic_cov, 'borders')
```

To allow a direct, position-matched comparison between conditions, we pooled the boundaries called in the Mock and infected samples into a single consensus set, as done in the original study.

```
merged_bounds <- GenomicRanges::reduce(c(bounds_mock, bounds_cov))
merged_bounds <- merged_bounds[seqnames(merged_bounds) == "chr14"]
head(merged_bounds)
```

GRanges object with 6 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr14	19250001-19260000	*
[2]	chr14	19460001-19470000	*
[3]	chr14	19790001-19800000	*
[4]	chr14	20000001-20010000	*
[5]	chr14	20690001-20700000	*
[6]	chr14	20740001-20750000	*

seqinfo: 25 sequences from an unspecified genome

```
ins_track_mock <- metadata(hic_mock)$insulation
ins_track_cov <- metadata(hic_cov)$insulation
```

We then used the `epiwraps` package to visualise insulation-score profiles centred on the consensus boundaries, extending 500 kb up- and downstream of each boundary.

```
ins_track_mock$score <- ins_track_mock$insulation
ins_track_cov$score <- ins_track_cov$insulation
tracks <- list(Mock = ins_track_mock, SARS_CoV_2 = ins_track_cov)
ins <- signal2Matrix(tracks, regions = merged_bounds, extend = 500000, w = 10000)
```

Computing signal from GRanges 'Mock'...

Computing signal from GRanges 'SARS_CoV_2'...

```
plotEnrichedHeatmaps(ins, scale_title = "Insulation Scores", use_raster = FALSE)
```

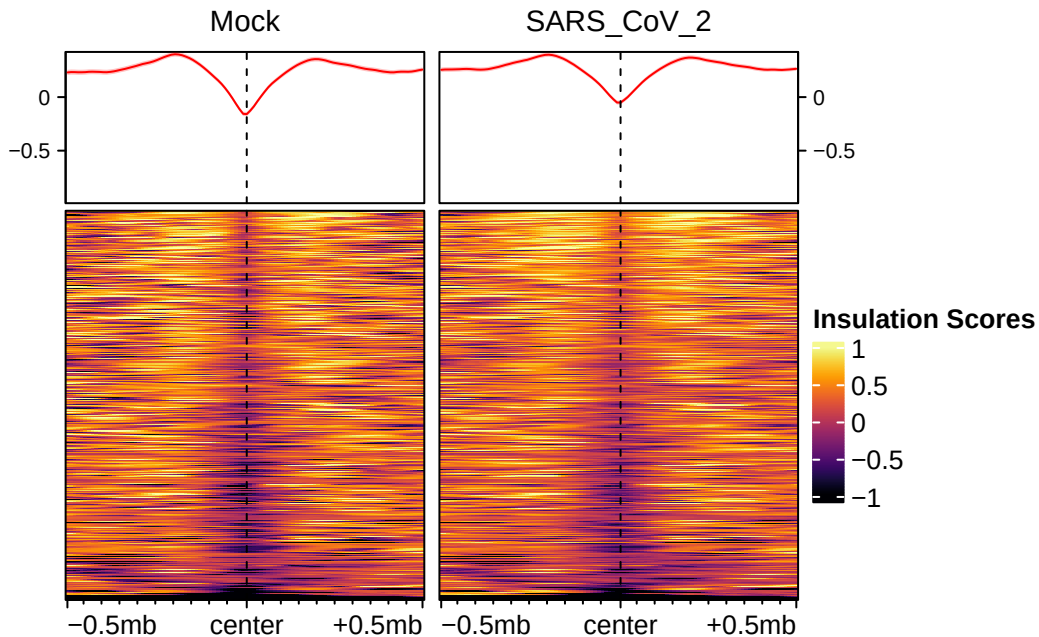


Figure 1: Insulation scores calculated TAD boundaries in both conditions.

We can see that although CoV-infected group shows a slightly higher insulation score at the consensus boundaries, this effect is rather mild, consistent with the original study (Fig.4d). This indicates that TAD boundary strength is largely preserved upon infection.

3. TADs and Intra-TAD Interaction

We then further obtained the TADs, defined as genomic intervals nested between the previously identified consensus boundaries.

```
# gaps between merged boundaries are TADs
merged_tads <- gaps(merged_bounds)
merged_tads <- merged_tads[strand(merged_tads) == "*"]
merged_tads <- merged_tads[seqnames(merged_tads) == "chr14"]
# filter out extremely small fragments (as we have resolution of 10000, use threshold 20000)
merged_tads <- merged_tads[width(merged_tads) >= 20000]
# get the GInteractions objects
gi_mock <- interactions(hic_mock)
gi_cov <- interactions(hic_cov)
# filter out very short distant interactions (< 15 kb)
gi_mock_sub <- gi_mock[pairdist(gi_mock) >= 15000]
gi_cov_sub <- gi_cov[pairdist(gi_cov) >= 15000]
```

To quantify the intra-TAD interactions, we defined a helper function to isolate interactions that occur strictly within the same TAD domain. We then computed the mean balanced contact score per TAD for each condition.

```
# func to map interactions inside TADs
get_intra_tad_means <- function(gi_filtered, tads) {

  hits1 <- findOverlaps(anchors(gi_filtered, type="first"), tads)
  hits2 <- findOverlaps(anchors(gi_filtered, type="second"), tads)

  # keep only interactions where both anchors found in the same TAD
  common_hits <- inner_join(
    as.data.frame(hits1),
    as.data.frame(hits2),
    by = "queryHits", suffix = c("1", "2")
  ) %>% filter(subjectHits1 == subjectHits2)

  # aggregate mean contact score per TAD

  df <- data.frame(
    tad_idx = common_hits$subjectHits1,
    contact_val = gi_filtered$balanced[common_hits$queryHits]
  )
}
```

```

df_mean <- df %>%
  group_by(tad_idx) %>%
  summarize(mean_contact = mean(contact_val, na.rm = TRUE))

return(df_mean)
}

mock_means <- get_intra_tad_means(gi_mock_sub, merged_tads)
cov_means <- get_intra_tad_means(gi_cov_sub, merged_tads)

```

To compare conditions, we then computed, for each TAD, the log2 fold-change of the mean contact score (SARS-CoV-2 relative to Mock).

```

tad_metrics <- data.frame(tad_id = 1:length(merged_tads)) %>%
  left_join(mock_means, by = c("tad_id" = "tad_idx")) %>%
  rename(Mock_Mean = mean_contact) %>%
  left_join(cov_means, by = c("tad_id" = "tad_idx")) %>%
  rename(Cov_Mean = mean_contact) %>%
  mutate(
    log2FC = log2((Cov_Mean) / (Mock_Mean))
  )

# save it as a col
mcols(merged_tads) <- tad_metrics

```

We then visualised the distribution of these log2FCs.

```

# Fig.4c: boxplot of intra-TAD interaction changes
ggplot(tad_metrics, aes(x = "Infection", y = log2FC)) +
  geom_violin(fill = "#E74C3C", width = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  labs(
    x = "",
    y = expression(log[2]~"Fold Change (SARS-CoV-2 / Mock)")
  ) +
  theme_classic(base_size = 14)

```

Warning: Removed 35 rows containing non-finite outside the scale range (``stat_ydensity()``).

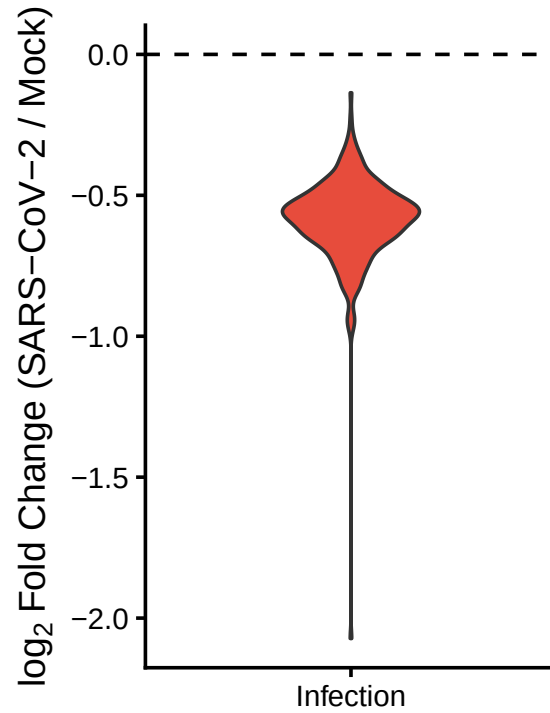


Figure 2: log₂FCs of intra-TAD Hi-C interactions.

It can be seen from the plot that SARS-CoV-2 infection reduces the intra-TAD contacts, which aligns with the findings of the paper.

4. Hi-C Matrices at a Specific TAD Domain

Following the original study (Fig.4a), we defined a specific TAD domain of interest on chromosome 14 and visualised Hi-C contacts at this region. We proceeded with bin size of 10kb, unlike in the paper, where the authors switched to a resolution of 5kb for this analysis. We calculated a differential contact matrix for this TAD region, which we visualised together with the contact matrices of both Mock and CoV groups.

```
# the target genomic coordinates
target_chr <- "chr14"
target_start <- 71400000
target_end <- 72600000
target_region <- paste0(target_chr, ":", target_start, "-", target_end)
target_gr <- GRanges(seqnames = target_chr, ranges = IRanges(target_start, target_end))
```

```

hic_mock_reg <- refocus(hic_mock, target_region)
hic_cov_reg <- refocus(hic_cov, target_region)

hic_div      <- divide(hic_cov_reg, by = hic_mock_reg, use.scores = "balanced")

# mock
p_matrix_mock <- plotMatrix(hic_mock_reg, use.scores = "balanced", scale = "log10", caption = "Mock",
  labs(title = "Mock", fill = "log10 Score", y = paste0(target_chr, ": 71,400,000-72,600,000"),
  theme(axis.text = element_blank(), axis.ticks = element_blank()))

# cov
p_matrix_cov <- plotMatrix(hic_cov_reg, use.scores = "balanced", scale = "log10", caption = "SARS-CoV-2",
  labs(title = "SARS-CoV-2", fill = "log10 Score") +
  theme(axis.title.y = element_blank(), axis.text = element_blank(), axis.ticks = element_blank()))

# log2 FC
p_matrix_l2fc <- plotMatrix(
  hic_div,
  use.scores = "balanced.l2fc",
  scale = "linear",
  cmap = bgrColors(), caption = FALSE,
  limits = c(-2, 2)
) +
  labs(title = "SARS-CoV-2/mock", fill = "log2 FC") +
  theme(axis.title.y = element_blank(), axis.text = element_blank(), axis.ticks = element_blank())

matrix_panels <- p_matrix_mock + p_matrix_cov + p_matrix_l2fc +
  plot_layout(ncol = 3)

matrix_panels

```

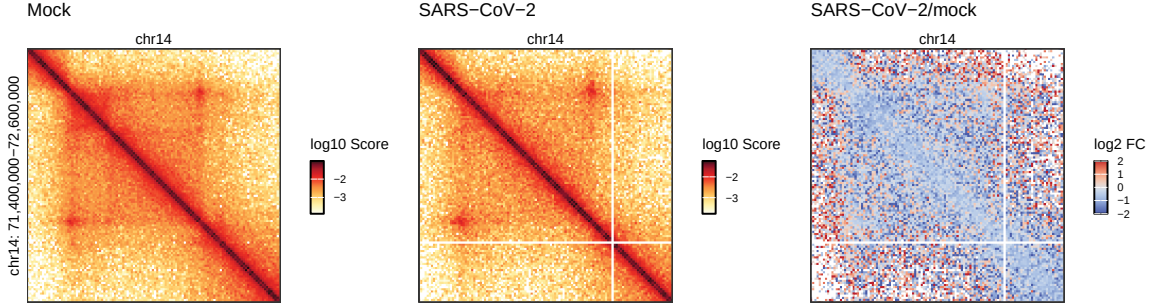


Figure 3: Hi-C matrices (bin size = 10 kb) at the indicated region (hg19) and log2FC after infection.

The patterns here indicate that while intra-TAD interactions tend to be reduced by infection of SARS-CoV-2, contacts outside of TADs (off-diagonal) are largely unaltered or even increased. This was demonstrated more comprehensively by the Aggregate Domain Analysis in the original paper (Fig.4b). Looking at the contact matrices of the two groups (Mock and CoV), we can also see that TAD-boundaries are not noticeably perturbed by infection, corroborating the results of the insulation score analysis.

5. Discussion

This part of the analysis shows that SARS-CoV-2 infection selectively weakens intra-TAD interactions, while leaving TAD boundaries and inter-TAD contacts largely intact.

It should be noted that our pipeline does not exactly replicate the original methods, since the paper did not fully specify several parameter choices. We were also unable to reproduce the Aggregate Domain Analysis due to a persistent function error that we could not resolve.

Despite these methodological differences, the trends we recovered from this part align well with the conclusions of the original paper.

Chromatin Loop Calling and Differential Analysis

Table of contents

Overview	1
Deviations from the paper	2
0. Data Preparation	2
1. Loop Calling with <code>cooltools dots</code>	3
1.1 Compute Expected Values	3
1.2 Call Loops at 10 kb Resolution	3
2. Build a Consensus Loop Set	4
3. Differential Loop Analysis	6
3.1 Prepare 40 kb Coolers	6
3.2 Extract Raw Contact Counts at Each Loop Locus	6
3.3 DESeq2 Differential Loop Testing	7
4. Visualisations	10
4.1 Volcano Plot	10
4.2 Loop-Size Distribution for Differential Loops	11
5. Aggregate Peak Analysis (APA)	13
5.1 Export Differential Loop Subsets	13
5.2 Run <code>coolpuppy</code>	14
5.3 Visualise APA Heatmaps	15
Discussion of our results	20

Overview

This notebook reproduces the **chromatin loop calling and enhancer–promoter contact analysis** from Wang, R., Lee, J.-H., Kim, J. *et al.* “SARS-CoV-2 restructures host chromatin architecture.” *Nature Microbiology* **8**, 679–694 (2023). <https://doi.org/10.1038/s41564-023-01344-8>

The paper shows that SARS-CoV-2 infection preferentially depletes cohesin from intra-TAD regions, weakening chromatin loops and reducing enhancer–promoter contacts at interferon response gene loci. We quantify and test these changes using a re-implementation of the HIC-CUPS loop-calling algorithm available in **cooltools**, followed by DESeq2 differential testing and Aggregate Peak Analysis (APA).

Deviations from the paper

The analysis shown in the paper looked at genome wide chromatin loops at 5kb and 10kb resolution. We focused our analysis on chromosome 14 and were limited to analysing loops at 10kb resolution.

0. Data Preparation

The same files and preprocessing steps as in the **tad analysis** were used here.

```
mkdir -p ./out

cooler merge ./out/merged_mock_HiC.mcool \
  ./GSE179184_RAW/mock_HiC_rep1.mcool::resolutions/10000 \
  ./GSE179184_RAW/mock_HiC_rep2.mcool::resolutions/10000

cooler merge ./out/merged_cov_HiC.mcool \
  ./GSE179184_RAW/cov_HiC_rep1.mcool::resolutions/10000 \
  ./GSE179184_RAW/cov_HiC_rep2.mcool::resolutions/10000

cooler balance ./out/merged_mock_HiC.mcool
cooler balance ./out/merged_cov_HiC.mcool

cooler zoomify -r 10000 \
  ./out/merged_mock_HiC.mcool \
  -o ./out/merged_mock_HiC_multi.mcool

cooler zoomify -r 10000 \
  ./out/merged_cov_HiC.mcool \
  -o ./out/merged_cov_HiC_multi.mcool
```

1. Loop Calling with `cooltools dots`

We use `cooltools dots` (the open2C re-implementation of HICCUPS (Rao *et al.*, 2014)) to detect pixels in the Hi-C contact matrix that are significantly enriched relative to a donut-shaped local expected background. Loop calling is performed on the merged replicate coolers to maximise read depth and sensitivity.

1.1 Compute Expected Values

`cooltools expected-cis` builds the background model of contact frequency as a function of genomic distance, required as input to `cooltools dots`.

```
# chr14 size in hg19 = 107,043,718 bp
echo -e "chr14\t0\t107043718\tchr14" > ./out/chr14_view.tsv

cooltools expected-cis \
  --nproc 4 \
  --view ./out/chr14_view.tsv \
  ./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
  -o ./out/expected_mock_10kb.tsv

cooltools expected-cis \
  --nproc 4 \
  --view ./out/chr14_view.tsv \
  ./out/merged_cov_HiC_multi.mcool::resolutions/10000 \
  -o ./out/expected_cov_10kb.tsv
```

1.2 Call Loops at 10 kb Resolution

Dots are called using the parameters specified in **4DN benchmarking protocol** (Oksuz *et al.*, 2021, *Nat. Methods*)

```
cooltools dots \
  --nproc 4 \
  --view ./out/chr14_view.tsv \
  --fdr 0.1 \
  --clustering-radius 39000 \
  --max-loci-separation 10000000 \
  --tile-size 5000000 \
  --max-nans-tolerated 4 \
  ./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
```

```

./out/expected_mock_10kb.tsv \
-o ./out/loops_mock_10kb.bedpe

cooltools dots \
  --nproc 4 \
  --view ./out/chr14_view.tsv \
  --fdr 0.1 \
  --clustering-radius 39000 \
  --max-loci-separation 10000000 \
  --tile-size 5000000 \
  --max-nans-tolerated 4 \
  ./out/merged_cov_HiC_multi.mcool::/resolutions/10000 \
  ./out/expected_cov_10kb.tsv \
  -o ./out/loops_cov_10kb.bedpe

```

2. Build a Consensus Loop Set

Here we look for loops that are present in both infected and mock groups.

```

read_loops <- function(path, tag) {
  raw <- read_tsv(path,
    comment = "#", col_types = cols(.default = "c"),
    show_col_types = FALSE
  )

  if (!("chrom1" %in% colnames(raw))) {
    colnames(raw)[1:6] <- c(
      "chrom1", "start1", "end1",
      "chrom2", "start2", "end2"
    )
  }

  raw %>%
    mutate(across(c(start1, end1, start2, end2), as.integer)) %>%
    filter(chrom1 == "chr14", chrom2 == "chr14") %>%
    mutate(
      mid1      = (start1 + end1) / 2L,
      mid2      = (start2 + end2) / 2L,
      loop_size = mid2 - mid1,

```

```

    source      = tag
  )
}

loops_mock <- read_loops("./out/loops_mock_10kb.bedpe", "mock")
loops_cov <- read_loops("./out/loops_cov_10kb.bedpe", "cov")

margin <- 10000

anchor_gr <- function(df, anchor) {
  if (anchor == 1) {
    GRanges(df$chrom1, IRanges(start = df$mid1, end = df$mid1))
  } else {
    GRanges(df$chrom2, IRanges(start = df$mid2, end = df$mid2))
  }
}

ov1 <- findOverlaps(anchor_gr(loops_mock, 1), anchor_gr(loops_cov, 1),
  maxgap = margin
)
ov2 <- findOverlaps(anchor_gr(loops_mock, 2), anchor_gr(loops_cov, 2),
  maxgap = margin
)

shared <- inner_join(as.data.frame(ov1), as.data.frame(ov2),
  by = c("queryHits", "subjectHits")
)

cov_unique <- loops_cov %>%
  filter(!(row_number() %in% unique(shared$subjectHits)))

loops_consensus <- bind_rows(loops_mock, cov_unique) %>%
  mutate(
    loop_id      = paste0("loop_", row_number()),
    loop_size_kb = loop_size / 1e3
  )

loops_consensus %>%
  select(chrom1, start1, end1, chrom2, start2, end2, loop_id) %>%
  write_tsv("./out/loops_consensus_chr14.bedpe", col_names = TRUE)

```

3. Differential Loop Analysis

Following the paper, we quantify raw Hi-C contact counts at 40 kb resolution of each called loop, then apply DESeq2 to test for differential analysis.

3.1 Prepare 40 kb Coolers

The individual replicate files contain 10 kb bins. We coarsen by a factor of 4 to reach 40 kb using `cooler coarsen`.

```
for SAMPLE in mock_HiC_rep1 mock_HiC_rep2 cov_HiC_rep1 cov_HiC_rep2; do
    cooler coarsen \
        --nproc 4 \
        -k 4 \
        ./GSE179184_RAW/${SAMPLE}.mcool::/resolutions/10000 \
        -o ./out/${SAMPLE}_40kb.cool
    echo "Coarsened: ${SAMPLE}_40kb.cool"
done
```

3.2 Extract Raw Contact Counts at Each Loop Locus

```
import cooler
import numpy as np
import pandas as pd

RESOLUTION = 40_000

SAMPLES = {
    "mock_rep1": "./GSE179184_RAW/mock_HiC_rep1.mcool::/resolutions/10000",
    "mock_rep2": "./GSE179184_RAW/mock_HiC_rep2.mcool::/resolutions/10000",
    "cov_rep1":  "./GSE179184_RAW/cov_HiC_rep1.mcool::/resolutions/10000",
    "cov_rep2":  "./GSE179184_RAW/cov_HiC_rep2.mcool::/resolutions/10000",
}

clrs = {name: cooler.Cooler(path) for name, path in SAMPLES.items()}

loops = pd.read_csv(
    "./out/loops_consensus_chr14.bedpe",
    sep="\t",
    header=None,
```

```

    names=["chrom1", "start1", "end1", "chrom2", "start2", "end2", "loop_id"],
)

loops = loops.drop(0)
loops[["start1", "end1", "start2", "end2"]] = loops[["start1", "end1", "start2", "end2"]].as

rows = []
for _, lp in loops.iterrows():
    m1 = int(((lp.start1 + lp.end1) // 2) // RESOLUTION * RESOLUTION)
    m2 = int(((lp.start2 + lp.end2) // 2) // RESOLUTION * RESOLUTION)

    row = {"loop_id": lp.loop_id}
    for name, clr in clr.items():
        try:
            pixel = clr.matrix(balance=False).fetch(
                (lp.chrom1, m1, m1 + RESOLUTION),
                (lp.chrom2, m2, m2 + RESOLUTION),
            )
            row[name] = int(np.nansum(pixel))
        except Exception:
            row[name] = 0
    rows.append(row)

count_df = pd.DataFrame(rows)
count_df.to_csv("./out/loop_raw_counts_40kb.tsv", sep="\t", index=False)

```

3.3 DESeq2 Differential Loop Testing

Here we perform DESeq2 differential analysis on the raw HI-C contact counts at 40kb resolution.

```

counts_raw <- read_tsv("./out/loop_raw_counts_40kb.tsv", show_col_types = FALSE)

count_mat <- counts_raw %>%
  select(mock_rep1, mock_rep2, cov_rep1, cov_rep2) %>%
  as.matrix()
rownames(count_mat) <- counts_raw$loop_id

keep <- rowSums(count_mat) > 0
count_mat <- count_mat[keep, , drop = FALSE]

```

```
col_data <- data.frame(
  condition = factor(c("Mock", "Mock", "SARS_CoV_2", "SARS_CoV_2"),
    levels = c("Mock", "SARS_CoV_2")
  ),
  row.names = c("mock_rep1", "mock_rep2", "cov_rep1", "cov_rep2")
)

dds <- DESeqDataSetFromMatrix(
  countData = count_mat,
  colData = col_data,
  design = ~condition
)
```

converting counts to integer mode

```
dds <- DESeq(dds, quiet = TRUE)

res <- results(
  dds,
  contrast = c("condition", "SARS_CoV_2", "Mock"),
  alpha = 0.1,
  pAdjustMethod = "BH",
  independentFiltering = FALSE
)

summary(res)
```

```
out of 3580 with nonzero total read count
adjusted p-value < 0.1
LFC > 0 (up)      : 30, 0.84%
LFC < 0 (down)    : 119, 3.3%
outliers [1]      : 0, 0%
low counts [2]    : 0, 0%
(mean count < 0)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

```
res_df <- as.data.frame(res) %>%
  rownames_to_column("loop_id") %>%
```



```

left_join(
  loops_consensus %>% select(loop_id, loop_size_kb),
  by = "loop_id"
) %>%
mutate(
  direction = case_when(
    !is.na(padj) & padj < 0.1 & log2FoldChange > 0 ~ "Strengthened",
    !is.na(padj) & padj < 0.1 & log2FoldChange < 0 ~ "Weakened",
    TRUE ~ "NS"
  ),
  direction = factor(direction, levels = c("Weakened", "NS", "Strengthened"))
)

n_weak <- sum(res_df$direction == "Weakened")
n_strong <- sum(res_df$direction == "Strengthened")
n_total <- nrow(res_df)

cat(sprintf(
  "Weakened loops:      %d  (0.1f%%)\n",
  n_weak, 100 * n_weak / n_total
))

```

Weakened loops: 119 (3.3%)

```

cat(sprintf(
  "Strengthened loops: %d  (0.1f%%)\n",
  n_strong, 100 * n_strong / n_total
))

```

Strengthened loops: 30 (0.8%)

```

cat(sprintf(
  "Not significant:    %d  (0.1f%%)\n",
  n_total - n_weak - n_strong,
  100 * (n_total - n_weak - n_strong) / n_total
))

```

Not significant: 3431 (95.8%)

The paper reported that genome wide 2.96% of loops were weakened and 4.7% were strengthened. We find that on chromosome 14, 3.3% are weakened and 0.8% are strengthened. This does not reflect the findings of the paper, but may be an artifact of looking solely at chromosome 14.

4. Visualisations

4.1 Volcano Plot

```
p_cutoff <- res_df %>%
  filter(!is.na(padj), padj < 0.1) %>%
  pull(pvalue) %>%
  max(na.rm = TRUE)

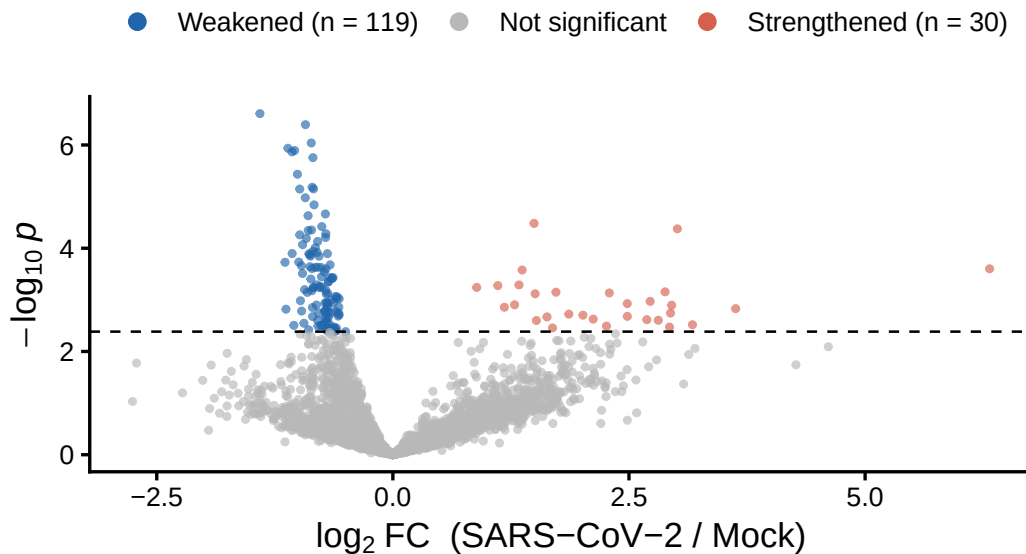
col_map <- c(
  Weakened = "#2166AC",
  NS = "grey72",
  Strengthened = "#D6604D"
)

label_map <- c(
  Weakened = sprintf("Weakened (n = %d)", n_weak),
  NS = "Not significant",
  Strengthened = sprintf("Strengthened (n = %d)", n_strong)
)

res_df %>%
  filter(!is.na(pvalue)) %>%
  arrange(direction) %>%
  ggplot(aes(
    x = log2FoldChange,
    y = -log10(pvalue),
    colour = direction
  )) +
  geom_point(size = 0.8, alpha = 0.65) +
  geom_hline(
    yintercept = -log10(p_cutoff),
    linetype = "dashed", colour = "black", linewidth = 0.45
```

```
) +
scale_colour_manual(values = col_map, labels = label_map) +
labs(
  x      = expression(log[2] ~ "FC (SARS-CoV-2 / Mock)",
  y      = expression(-log[10] ~ italic(p)),
  colour = NULL,
  title  = "Differential chromatin loops - chr14"
) +
guides(colour = guide_legend(override.aes = list(size = 2.5, alpha = 1))) +
theme_classic(base_size = 13) +
theme(legend.position = "top")
```

Differential chromatin loops – chr14



The observed fold-changes across both weakened and strengthened loops are comparable to the changes reported in the paper (extended data figure 8). However, the ratio of strengthened to weakened loops varies wildly between this analysis and the papers findings as previously noted.

4.2 Loop-Size Distribution for Differential Loops

```
diff_loops <- res_df %>%
  filter(direction != "NS", !is.na(loop_size_kb)) %>%
```

```

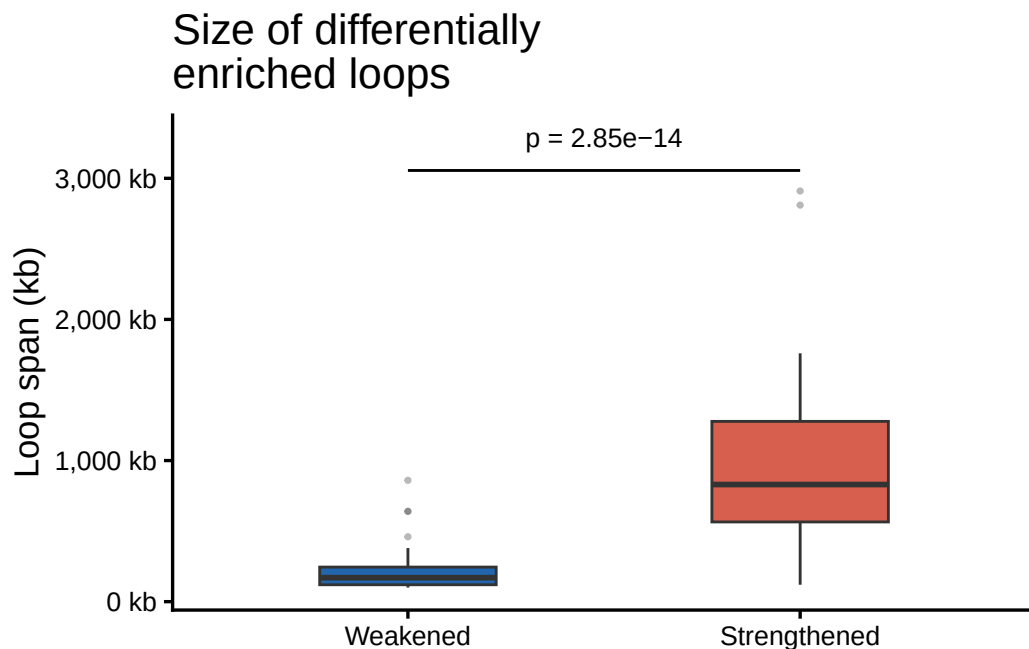
droplevels()

wt <- wilcox.test(
  loop_size_kb ~ direction,
  data = diff_loops,
  exact = FALSE
)

y_bar <- max(diff_loops$loop_size_kb, na.rm = TRUE)

ggplot(diff_loops, aes(x = direction, y = loop_size_kb, fill = direction)) +
  geom_boxplot(
    outlier.size = 0.6, outlier.alpha = 0.35,
    width = 0.45, linewidth = 0.5
  ) +
  annotate("segment",
    x = 1, xend = 2,
    y = y_bar * 1.05, yend = y_bar * 1.05,
    colour = "black", linewidth = 0.5
  ) +
  annotate("text",
    x = 1.5, y = y_bar * 1.13, hjust = 0.5, size = 3.5,
    label = sprintf("p = %.2e", wt$p.value)
  ) +
  scale_fill_manual(
    values = c(Weakened = "#2166AC", Strengthened = "#D6604D"),
    guide = "none"
  ) +
  scale_y_continuous(labels = comma_format(suffix = " kb")) +
  labs(
    x = NULL,
    y = "Loop span (kb)",
    title = "Size of differentially\nenriched loops"
  ) +
  theme_classic(base_size = 13)

```



The paper reports that virus-weakened loops are predominantly short, while strengthened loops are long and the range of different lengths is larger in strengthened loops. This is consistent with our analysis of chromosome 14.

5. Aggregate Peak Analysis (APA)

We perform aggregate peak analysis on all conditions and loop change types using `coolpuppy`.

5.1 Export Differential Loop Subsets

```
write_bedpe <- function(ids, file) {
  loops_consensus %>%
    filter(loop_id %in% ids) %>%
    select(chrom1, start1, end1, chrom2, start2, end2, loop_id) %>%
    write_tsv(file, col_names = TRUE)
  cat(sprintf("Wrote %d loops to %s\n", length(ids), file))
}
```

```

write_bedpe(
  filter(res_df, direction == "Weakened") %>% pull(loop_id),
  "./out/loops_weakened.bedpe"
)

```

Wrote 119 loops to ./out/loops_weakened.bedpe

```

write_bedpe(
  filter(res_df, direction == "Strengthened") %>% pull(loop_id),
  "./out/loops_strengthened.bedpe"
)

```

Wrote 30 loops to ./out/loops_strengthened.bedpe

5.2 Run coolpuppy

```

# consensus loops
coolpup.py \
  --expected ./out/expected_mock_10kb.tsv \
  --view ./out/chr14_view.tsv \
  --flank 100000 \
  ./out/merged_mock_HiC_multi.mcool::resolutions/10000 \
  ./out/loops_consensus_chr14.bedpe \
  --outname ./out/apa_mock_all.clpy

coolpup.py \
  --expected ./out/expected_cov_10kb.tsv \
  --view ./out/chr14_view.tsv \
  --flank 100000 \
  ./out/merged_cov_HiC_multi.mcool::resolutions/10000 \
  ./out/loops_consensus_chr14.bedpe \
  --outname ./out/apa_cov_all.clpy

# weakened
coolpup.py \
  --expected ./out/expected_mock_10kb.tsv \
  --view ./out/chr14_view.tsv \
  --flank 100000 \
  ./out/merged_mock_HiC_multi.mcool::resolutions/10000 \

```

```

./out/loops_weakened.bedpe \
--outname ./out/apa_mock_weakened.clpy

coolpup.py \
--expected ./out/expected_cov_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_cov_HiC_multi.mcool::/resolutions/10000 \
./out/loops_weakened.bedpe \
--outname ./out/apa_cov_weakened.clpy

# strengthened
coolpup.py \
--expected ./out/expected_mock_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_mock_HiC_multi.mcool::/resolutions/10000 \
./out/loops_strengthened.bedpe \
--outname ./out/apa_mock_strengthened.clpy

coolpup.py \
--expected ./out/expected_cov_10kb.tsv \
--view ./out/chr14_view.tsv \
--flank 100000 \
./out/merged_cov_HiC_multi.mcool::/resolutions/10000 \
./out/loops_strengthened.bedpe \
--outname ./out/apa_cov_strengthened.clpy

```

5.3 Visualise APA Heatmaps

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import re
import h5sparse

```

```

/home/louis/miniconda3/lib/R/library/reticulate/python/rpytools/loader.py:120: UserWarning: 
11-30. Refrain from using this package or pin to Setuptools<81.
    return _find_and_load(name, import_)

```

```

import os
from matplotlib.colors import LogNorm

def load_pileup_df(filename, quaich=False, skipstripes=False):
    """
    Loads a dataframe saved using `save_pileup_df`

    Parameters
    -----
    filename : str
        File to load from.
    quaich : bool, optional
        Whether to assume standard quaich file naming to extract sample name and bedname.
        The default is False.

    Returns
    -----
    annotation : pd.DataFrame
        Pileups are in the "data" column, all metadata in other columns

    """
    with h5sparse.File(filename, "r", libver="latest") as f:
        metadata = dict(zip(f["attrs"].attrs.keys(), f["attrs"].attrs.values()))
        dstore = f["data"]
        data = []
        for chunk in dstore.iter_chunks():
            chunk = dstore[chunk]
            data.append(chunk)
        annotation = pd.read_hdf(filename, "annotation")
        annotation["data"] = data
        vertical_stripe = []
        horizontal_stripe = []
        coordinates = []
        if not skipstripes:
            try:
                for i in range(len(data)):
                    vstripe = "vertical_stripe_" + str(i)
                    hstripe = "horizontal_stripe_" + str(i)
                    coords = "coordinates_" + str(i)
                    vertical_stripe.append(f[vstripe][:].toarray())
                    horizontal_stripe.append(f[hstripe][:].toarray())
                    coordinates.append(f[coords][:].astype("U13"))

```



```

        annotation["vertical_stripe"] = vertical_stripe
        annotation["horizontal_stripe"] = horizontal_stripe
        annotation["coordinates"] = coordinates
    except KeyError:
        pass
for key, val in metadata.items():
    if key != "version":
        annotation[key] = val
if quaich:
    basename = os.path.basename(filename)
    sample, bedname = re.search(
        "^(.*)-(?:[0-9]+)_over_(.*)_(?:[0-9]+-shifts|expected).*\\.clpy", basename
    ).groups()
    annotation["sample"] = sample
    annotation["bedname"] = bedname
return annotation

def load_pileup(path):
    df = load_pileup_df(path)
    return np.nanmean(np.stack(df["data"].values), axis=0)

def plot_apa_pair(path_mock, path_cov, title_mock, title_cov):
    mat_mock = load_pileup(path_mock)
    mat_cov = load_pileup(path_cov)

    bin_size_kb = (2 * 100_000) / (mat_mock.shape[0] - 1) / 1000

    combined = np.concatenate([mat_mock.flatten(), mat_cov.flatten()])
    valid_vals = combined[(~np.isnan(combined)) & (combined > 0)]
    vmin = np.nanpercentile(valid_vals, 5)
    vmax = np.nanpercentile(valid_vals, 95)

    fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))

    n = mat_mock.shape[0]
    centre = n // 2
    flank_kb = centre * bin_size_kb
    extent = [-flank_kb, flank_kb, flank_kb, -flank_kb]

    for ax, mat, title in zip(axes, [mat_mock, mat_cov], [title_mock, title_cov]):
        im = ax.matshow(mat, cmap="YlOrRd", norm=LogNorm(vmin=vmin, vmax=vmax), extent=extent)

```

```

ax.set_title(title, fontsize=11, pad=12, fontweight="bold")
ax.set_xlabel("Distance from center (kb)", fontsize=9)
ax.xaxis.set_label_position('bottom')
ax.xaxis.tick_bottom()

ax.set_xticks([-flank_kb, 0, flank_kb])
ax.set_yticks([-flank_kb, 0, flank_kb])

fig.subplots_adjust(left=0.08, right=0.82, bottom=0.15, top=0.85, wspace=0.35)

cbar_ax = fig.add_axes([0.86, 0.22, 0.025, 0.55])
cbar = fig.colorbar(im, cax=cbar_ax)
cbar.set_label("Enrichment (O/E)", fontsize=10, labelpad=8)

return fig

```

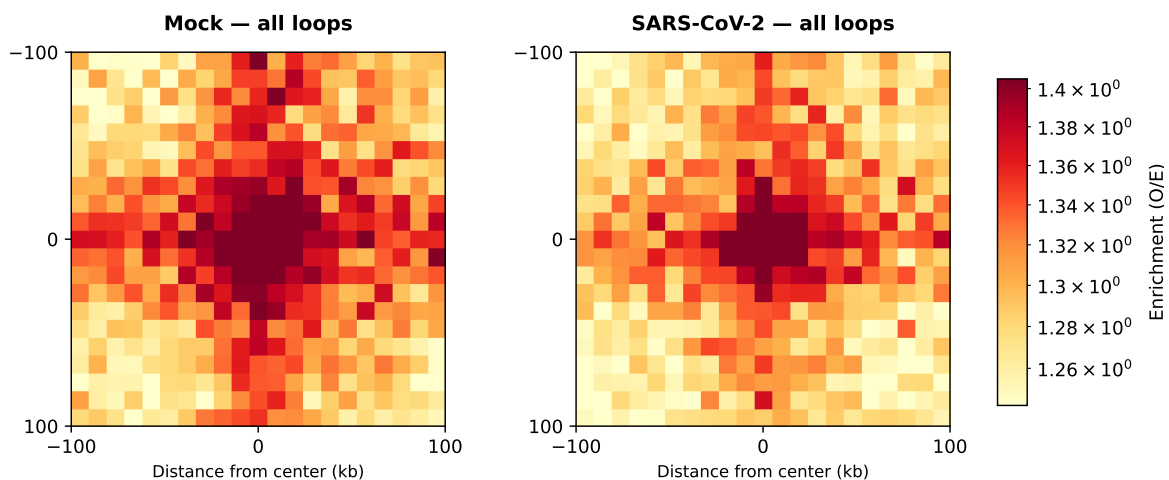
```

plot_apa_pair(
    "./out/apa_mock_all.clpy", "./out/apa_cov_all.clpy",
    "Mock - all loops", "SARS-CoV-2 - all loops",
)

```

<Figure size 900x450 with 3 Axes>

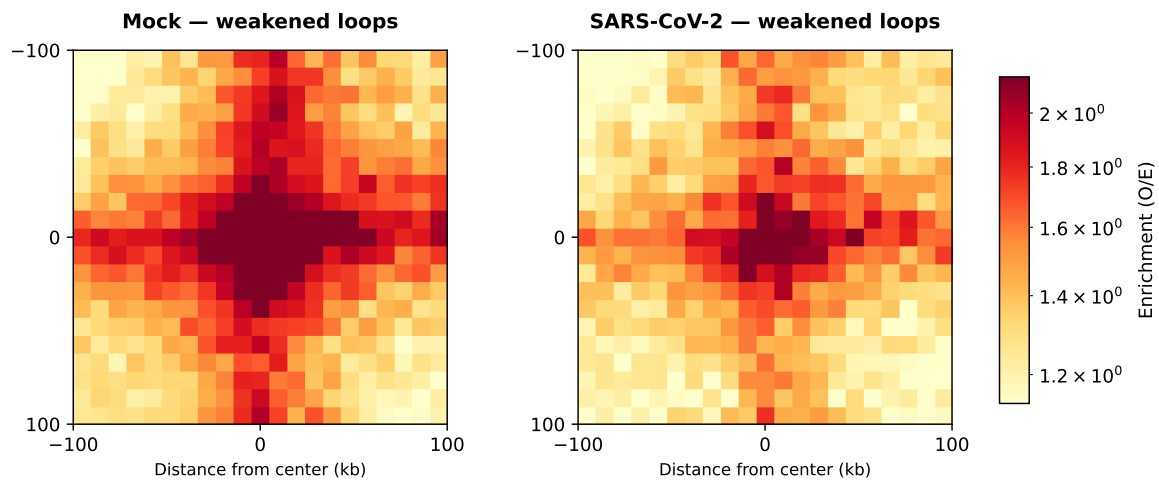
```
plt.show()
```



```
plot_apa_pair(
    "./out/apa_mock_weakened.clpy", "./out/apa_cov_weakened.clpy",
    "Mock - weakened loops", "SARS-CoV-2 - weakened loops",
)
```

<Figure size 900x450 with 3 Axes>

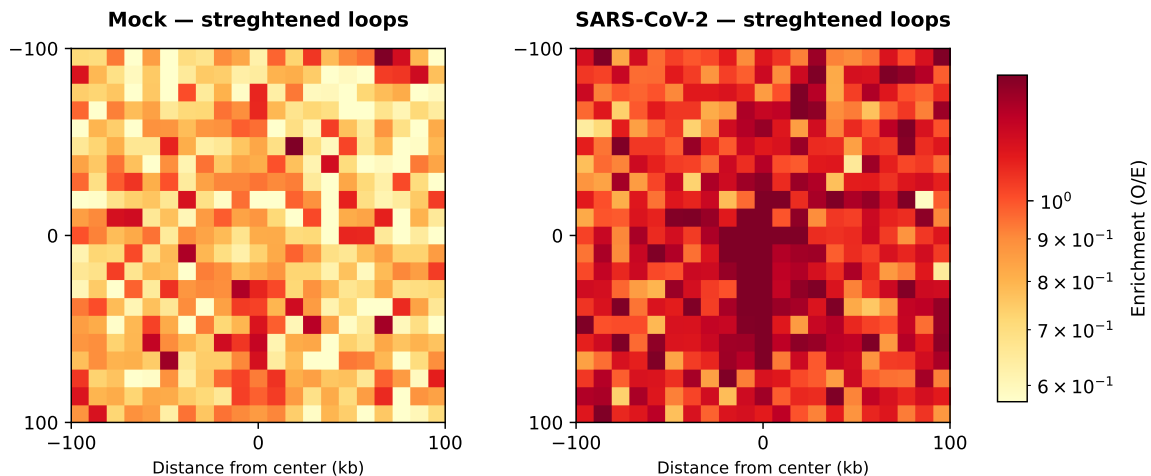
```
plt.show()
```



```
plot_apa_pair(
    "./out/apa_mock_strengthened.clpy", "./out/apa_cov_strengthened.clpy",
    "Mock - strengthened loops", "SARS-CoV-2 - strengthened loops",
)
```

<Figure size 900x450 with 3 Axes>

```
plt.show()
```



The enrichment score of all loops in mock and infected groups is comparable. Loops of the weakened category have a lower enrichment score in the infected group than in the mock group. These results match the observations in the paper.

In the APA heatmap for strengthened loops, no centralized signal is visible. While the enrichment score is higher in the infected group, which is in line with the paper's observations, the low sample size of strengthened loops makes this result unreliable.

Discussion of our results

Since our analysis was constrained to chromosome 14 and to the analysis of loops at 10 kb resolution, the sample size of loops for our analysis was lower than in the paper. Our results do not directly replicate the results in the paper, in particular we did not find that more loops were strengthened than weakened, instead finding the reverse. However the reported fold changes, as well as length distributions for strengthened and weakened loops aligned with our findings. Our APA analysis also mostly supports the findings in the paper.

In conclusion we were able to show that some loops are weakened or strengthened in the infected group, but could not replicate the exact quantitative findings of the paper we tried to reproduce. This can in large parts be explained by our analysis focusing only on a subset of the experimental data.